

Resolucion del Practico 8

Testing de Software 2024

Índice

Introduccion	3
1. Ejercicio 1: NodeCachingLinkedList con Randoop	3
1.1. Objetivo	3
1.2. Procedimiento	3
1.3. Defectos detectados por Randoop	4
1.4. Comandos	4
1.5. Resultados	4
1.6. Nota tecnica	5
2. Ejercicio 2: fileExample con Randoop y EvoSuite	5
2.1. Analisis del codigo	5
2.2. Randoop	6
2.3. EvoSuite	6
2.4. Comparacion	7
3. Ejercicio 3: Mecanismos de Randoop para mejorar escenarios como el Ejercicio 2	7
3.1. Punto de partida	7
3.2. Mecanismos disponibles	8
3.3. Estrategia recomendada para fileExample	9
3.4. Conclusion	10
4. Ejercicio 4: tests para IPBlacklist.login con EasyMock	10
4.1. Que hace IPBlacklist	10
4.2. Por que necesitamos un mock	10
4.3. Implementacion	10
4.4. Que valida cada test	11

4.5. Ejecucion	11
5. Ejercicio 5: fail2ban.Server con Randoop, EvoSuite y jqwik	12
5.1. Tareas pedidas	12
5.2. (a) Implementacion de <code>repOK()</code>	12
5.3. (b) Randoop, analisis y depuracion	12
5.4. (c) EvoSuite, comparacion con Randoop	13
5.5. (d) Usar <code>repOK()</code> para depurar con EvoSuite	14
5.6. (e) Propiedad <code>jqwik</code> sobre <code>update</code> con un unico generador	14
5.7. Resumen del ejercicio 5	16
Conclusiones	16

Introduccion

Este documento contiene la resolucion del *Practico 8* de Testing de Software, que integra tres ejes complementarios al testing manual:

- **Generacion automatica de tests** con dos herramientas:
 - **Randoop**: generacion aleatoria guiada por feedback, partiendo de la API publica de la clase bajo prueba.
 - **EvoSuite**: generacion guiada por busqueda evolutiva, optimizando una funcion objetivo basada en cobertura de ramas y mutacion.
- **Mocking** con EasyMock para aislar la clase bajo prueba de sus colaboradores (LoginService en el ejercicio 4).
- **Property-Based Testing** con jqwik, usando un unico generador para ejercitar el metodo `Server.update()`.

Convenciones. Para que el entorno sea reproducible, todos los comandos asumen ejecucion desde `tp8/assignment-8-rodeghiero`. La generacion con Randoop se hace con Java 17; la generacion con EvoSuite con Java 11 (su runtime esta pensado para JDK 8/11).

Scripts auxiliares incluidos.

```
# gen-randoop.sh <clase> <segundos> <outputlimit> <testsperfile>
./gen-randoop.sh assignment8_exercises.ncl.NodeCachingLinkedList
    20 400 200

# gen-evo.sh <clase> <segundos>
./gen-evo.sh assignment8_exercises.fail2ban.Server 30
```

1. Ejercicio 1: NodeCachingLinkedList con Randoop

1.1. Objetivo

Aplicar testing automatico sobre `NodeCachingLinkedList` usando Randoop, analizar las fallas que surjan, corregirlas en el codigo fuente, regenerar la suite como tests de regresion y reportar metricas de cobertura (JaCoCo) y de mutacion (PIT).

1.2. Procedimiento

1. Compilar el proyecto y generar la suite con Randoop.
2. Analizar los `ErrorTest` (casos que disparan crashes inesperados).

3. Corregir los defectos en el código.
4. Regenerar la suite, esta vez como `RegressionTest` (la suite queda verde y se conserva como batería de regresión).
5. Medir cobertura y mutación.

1.3. Defectos detectados por Randoop

Randoop expuso dos problemas en `NodeCachingLinkedList`:

1. **`maximumCacheSize` sin inicializar.** El constructor dejaba el campo en 0, lo que hacía que `isCacheFull()` fuera siempre `true` y la cache nunca se usara. Se corrigió agregando `maximumCacheSize = DEFAULT_MAXIMUM_CACHE_SIZE` en el constructor.
2. **`repOK()` sin implementar.** Devolvía siempre `false`, lo que invalidaba toda instancia desde el punto de vista del invariante. Se implementó completo y se anotó con `@randoop.CheckRep` para que Randoop lo evalúe durante la generación.

1.4. Comandos

```
# Compilar
JAVA_HOME=$(/usr/libexec/java_home -v 17) mvn -q -DskipTests
  compile

# Generar suite con Randoop
./gen-randoop.sh assignment8_exercises.ncl.NodeCachingLinkedList
  20 400 200

# Ejecutar la suite
JAVA_HOME=$(/usr/libexec/java_home -v 17) mvn -q test

# Analisis de mutacion con PIT
JAVA_HOME=$(/usr/libexec/java_home -v 17) \
  mvn -q org.pitest:pitest-maven:mutationCoverage
```

1.5. Resultados

Suite final:

- 217 tests ejecutados, 0 fallos, 0 errores.

Cobertura JaCoCo sobre `NodeCachingLinkedList`:

- Ramas cubiertas: 25 / 70.
- **Branch coverage: 35,71 %.**

Mutacion con PIT:

- Mutantes generados: 102.
- Mutantes muertos: 39.
- Mutantes sin cobertura: 44.
- **Mutation score: 38 %.**
- **Test strength: 67 %** (calculado sobre los mutantes que efectivamente cubre la suite).

Lectura. Randoop detecta una porcion razonable de los mutantes que ejecuta (test strength alto), pero la baja cobertura de ramas limita el mutation score global. Esto es tipico de la generacion aleatoria pura: explora la API publica pero no construye secuencias complejas que toquen las ramas mas profundas (por ejemplo, llenar y vaciar la cache muchas veces).

1.6. Nota tecnica

Para que las herramientas funcionen en JDK 17 fue necesario actualizar:

- `maven-surefire-plugin` a 3.2.5.
- `jacoco-maven-plugin` a 0.8.11.

Tambien se ajusto `gen-randoop.sh` para acotar la salida y obtener una suite manejable.

2. Ejercicio 2: `fileExample` con Randoop y EvoSuite

2.1. Analisis del codigo

La clase `fileExample.checkContent()` tiene una sola operacion publica, pero concentra varias dependencias *fuertes* con el entorno de ejecucion:

1. Lee un nombre de archivo desde `System.in`.
2. Cierra el `Scanner` asociado a consola (que tambien cierra `System.in`).
3. Si el archivo no existe, retorna `false`.
4. Si existe, lee la primera linea del archivo.
5. Compara esa linea con la fecha actual formateada con `DateFormat.SHORT`.

Esto deja a cualquier test automatico expuesto a cuatro fuentes de no determinismo: **entrada estandar, sistema de archivos, reloj y locale.**

2.2. Randoop

Comando:

```
./gen-randoop.sh assignment8_exercises.fileContents.fileExample
25 300 200
```

Salida:

- RegressionTest0 con un unico test.
- Sin ErrorTest.
- El test verifica la NoSuchElementException que aparece cuando no hay datos en System.in.

Ejecucion segura (evitando que el test se bloquee esperando entrada estandar):

```
JAVA_HOME=$(/usr/libexec/java_home -v 17) \  
mvn -q -Djacoco.skip=true -DforkCount=0 \  
-Dtest=assignment8_exercises.fileContents.RegressionTest test \  
< /dev/null
```

Resultado: 1 test, 0 fallos, 0 errores.

Interpretacion. Randoop explora muy poco comportamiento funcional aqui: su generacion aleatoria no puede “fabricar” un archivo en disco ni un **stdin** con contenido razonable, asi que se queda en el caso trivial de excepcion por entrada vacia.

2.3. EvoSuite

Comando (Java 11 para que funcione el runtime de EvoSuite):

```
JAVA_HOME=$(/usr/libexec/java_home -v 11) PATH=$JAVA_HOME/bin: \  
$PATH \  
./gen-evo.sh assignment8_exercises.fileContents.fileExample \  
20
```

Salida:

- fileExample_ESTest con 3 tests.
- fileExample_Failed_ESTest con casos asociados a violaciones / errores.

Cobertura reportada por EvoSuite durante la generacion:

- Cobertura promedio: **60 %**.
- Line: 53 %.
- Branch: 60 %.
- Weak mutation: 10 %.

EvoSuite tambien detecto excepciones no declaradas (e.g. `No line found`), lo cual es coherente con un `Scanner.nextLine()` que no valida disponibilidad previa.

Limitacion observada. Al ejecutar los tests generados sobre JDK 17, el runner de EvoSuite no encuentra `tools.jar` (estaba pensado para JDK 8). Por eso la *comparacion* se hace sobre las metricas reportadas por EvoSuite en la fase de generacion, no sobre la corrida posterior.

2.4. Comparacion

	Randoop	EvoSuite
Tests generados	1	3 (+ Failed suite)
Cobertura branch	—	60 %
ErrorTest	0	varios
Costo de integracion	bajo (Java 17)	alto (Java 11, runtime fragil)

Conclusion. Para `fileExample`, EvoSuite es mas expresivo en la generacion (mas casos, deteccion de excepciones no declaradas). Randoop es mas practico de poner en marcha. Ninguno reemplaza al otro: son trade-offs distintos entre profundidad de exploracion y facilidad de integracion.

3. Ejercicio 3: Mecanismos de Randoop para mejorar escenarios como el Ejercicio 2

3.1. Punto de partida

`fileExample.checkContent()` reúne dificultades clásicas para la generacion automatica: dependencia de `System.in`, lectura de archivos reales, fecha del sistema, locale, y un efecto lateral sobre el estado global (cierre de `System.in`). Con la configuracion por defecto, Randoop rara vez logra tests utiles en estos casos.

3.2. Mecanismos disponibles

(1) Setup/teardown inyectado

```
--junit-before-each    --junit-after-each
--junit-before-all     --junit-after-all
```

Permite *insertar* bloques de código en cada test generado: recrear `System.in`, crear un archivo temporal controlado, cargar contenido esperado, limpiar al final.

(2) Fijar propiedades del sistema

```
--system-props=user.language=en
--system-props=user.country=US
--system-props=user.timezone=UTC
```

Estabiliza el formato de fecha y locale, eliminando ese grado de libertad de los tests.

(3) Inyectar literales

```
--literals-file=...    --literals-level=...
```

Ayuda a Randoop a elegir valores útiles (rutas, identificadores, cadenas) en lugar de limitarse a los literales inferidos del bytecode.

(4) Filtrar métodos

```
--methodlist=...      --omitmethods=...
```

Concentra la generación en APIs relevantes y evita ruido (e.g. omitir métodos que cierran streams o que dependen de IO interactiva).

(5) Clasificación de excepciones

```
--checked-exception=...  --unchecked-exception=...
--npe-on-null-input=...  --npe-on-non-null-input=...
```

Cambia como se clasifica cada caso: `ErrorTest` (bug a investigar), `RegressionTest` (test de regresión), o `INVALID` (descartado).

(6) Presupuesto y tamaño de la suite

```
--timelimit    --inputlimit    --outputlimit
--maxsize      --small-tests
```

Priorizar tests legibles y evitar que la suite se vuelva inmanejable.

(7) Bloqueos y timeouts

```
--usethreads=true    --timeout=2000
```

Clave cuando algun camino puede quedar esperando entrada indefinidamente (como con `System.in`).

(8) Captura de salida

```
--capture-output=true
```

Reduce el ruido cuando el codigo bajo prueba imprime por `stdout/stderr`.

(9) Contratos con `@CheckRep`

Anotando un metodo con `@randoop.CheckRep`, Randoop lo evalua durante la generacion como invariante. Si se rompe, reporta el caso como `ErrorTest`. Util en estructuras con estado interno (e.g. `NodeCachingLinkedList.repOK()`).

3.3. Estrategia recomendada para `fileExample`

1. `-junit-before-each` para setear `System.in` y preparar un archivo temporal de prueba.
2. `-junit-after-each` para restaurar el estado.
3. `-system-props` para fijar locale y timezone.
4. `-small-tests`, `-maxsize`, `-outputlimit` para mantener la suite acotada.
5. `-timeout` por test para cortar bloqueos de `stdin`.

Comando ejemplo:

```
java -classpath target/classes:libs/randoop-all-3.0.8.jar randoop
    .main.Main gentests \
    --testclass=assignment8_exercises.fileContents.fileExample \
    --timelimit=60 --small-tests=true --maxsize=40 --outputlimit
        =300 \
    --junit-output-dir=src/test/java \
    --junit-package-name=assignment8_exercises.fileContents \
    --junit-before-each=src/test/resources/randoop/beforeEach.txt
        \
    --junit-after-each=src/test/resources/randoop/afterEach.txt \
    --system-props=user.language=en \
    --system-props=user.country=US \
```

```
--system-props=user.timezone=UTC \  
--capture-output=true --usethreads=true --timeout=2000
```

3.4. Conclusion

Por defecto Randoop rinde mal sobre código fuertemente acoplado al entorno. Combinando hooks de setup/teardown, propiedades del sistema y control de presupuesto, la utilidad y estabilidad de los tests generados mejoran significativamente.

4. Ejercicio 4: tests para `IPBlacklist.login` con EasyMock

4.1. Que hace `IPBlacklist`

`IPBlacklist` delega la autenticación en un `LoginService` y, sobre esa base, agrega política de bloqueo:

1. Registra el último IP que intento loguearse.
2. Cuenta intentos fallidos *consecutivos* del mismo IP.
3. Agrega el IP a la blacklist cuando acumula 3 intentos fallidos consecutivos.

4.2. Por que necesitamos un mock

El método `login(...)` depende del resultado de `service.login(...)`. Sin un `LoginService` real (con base de datos, hashing, etc.), no se puede observar el comportamiento de la política. Usamos `EasyMock` para construir un doble que *programamos* para devolver `false` la cantidad de veces que nos interese.

4.3. Implementación

```
@Before  
public void setUp() {  
    ipblacklist = new IPBlacklist();  
    service = createMock(LoginService.class);  
    ipblacklist.setService(service);  
}  
  
@Test  
public void shouldBlacklistIpAfterThreeConsecutiveFailedAttempts  
() {  
    String ip = "192.168.0.10";
```

```

    String user = "usuario";
    String password = "clave";
    String hash = Utils.getPasswordHashMD5(password);

    expect(service.login(ip, user, hash)).andReturn(false).times
        (3);
    replay(service);

    assertFalse(ipblacklist.login(ip, user, password));
    assertFalse(ipblacklist.login(ip, user, password));
    assertFalse(ipblacklist.login(ip, user, password));

    assertTrue(ipblacklist.blacklisted(ip));
    verify(service);
}

@Test
public void
    shouldNotBlacklistIpWhenFailedAttemptsAreLessThanThree() {
    String ip = "192.168.0.20";
    // ... mock: expect login(...).andReturn(false).times(2);
    // 2 intentos fallidos: NO debe estar en blacklist
    assertFalse(ipblacklist.blacklisted(ip));
}

```

4.4. Que valida cada test

- **shouldBlacklistIpAfterThreeConsecutiveFailedAttempts:** tras programar el mock para devolver `false` tres veces, se invocan tres `login(...)` consecutivos. Se verifica que el IP queda en la blacklist y que el mock se invoco exactamente las veces esperadas (`verify(service)`).
- **shouldNotBlacklistIpWhenFailedAttemptsAreLessThanThree:** solo dos intentos fallidos; el IP *no* debe quedar en blacklist.

4.5. Ejecucion

```

JAVA_HOME=$(/usr/libexec/java_home -v 17) \
    mvn -Djacoco.skip=true -Dtest=IPBlacklistTest test

```

Resultado: ambos tests pasan correctamente.

5. Ejercicio 5: fail2ban.Server con Randoop, EvoSuite y jqwik

5.1. Tareas pedidas

1. Implementar `repOK()`.
2. Generar tests con Randoop (30 s), analizarlos y depurar.
3. Generar tests con EvoSuite (30 s), comparar con Randoop.
4. Explicar como aprovechar `repOK()` para depurar con EvoSuite.
5. Escribir una propiedad jqwik para `update` con un *unico* generador.

5.2. (a) Implementacion de `repOK()`

Validaciones incluidas en `Server.repOK()`:

- `expirationTime` no nulo y > 0 .
- `time` no nulo.
- `exceptions` y `bans` no nulos.
- `exceptions.repOK()` y `bans.repOK()` verdaderos.
- Ninguna IP puede estar simultaneamente en `exceptions` y `bans`.
- Si `lastUpdate != null`, todos los bans deben tener `expires > lastUpdate`.

Se anoto con `@CheckRep` y se completaron tambien los invariantes auxiliares en `SinglyLinkedList` y `StrictlySortedSinglyLinkedList`.

5.3. (b) Randoop, analisis y depuracion

Corrida base.

```
./gen-randoop.sh assignment8_exercises.fail2ban.Server 30 500 200
```

Salida: `failing inputs=0`, sin `ErrorTest`, suite de regresion de 250 tests.

Problema observado. Las aserciones generadas incluian valores que dependen del reloj (e.g. `lastUpdate`, embebido en `toString()`). Al re-ejecutar, aparecen fallas intermitentes (tests *flaky*) que no representan defectos reales.

Suite estable. Para obtener metricas confiables se regenero con opciones mas estrictas:

```
java -classpath target/classes:libs/randoop-all-3.0.8.jar randoop
    .main.Main gentests \
    --testclass=assignment8_exercises.fail2ban.Server \
    --timelimit=30 --outputlimit=500 --testsperfile=200 --small-
        tests=true \
    --junit-output-dir=src/test/java \
    --junit-package-name=assignment8_exercises.fail2ban \
    --forbid-null=true --null-ratio=0 \
    --npe-on-null-input=INVALID --npe-on-non-null-input=ERROR \
    --no-regression-assertions=true --ignore-flaky-tests=true
```

Resultado: failing inputs=0, 174 tests verdes.

Defectos corregidos durante la depuracion (en `Server.java`, `SinglyLinkedList.java`, `StrictlySortedSinglyLinkedList.java`):

- Correccion en `SinglyLinkedList.remove(...)`: eliminacion correcta y decremento de `size`.
- Validaciones de null en operaciones publicas para evitar NPEs innecesarios.
- `StrictlySortedSinglyLinkedList.removeFromIP(...)` devuelve false cuando no remueve nada.
- Robustez general en recorridos e inserciones.

Cobertura y mutacion (Randoop)

- Branch coverage (JaCoCo, sobre `Server`): $2/52 = 3,85\%$.
- Mutation score (PIT, sobre `Server`): $2/54 = 4\%$.

5.4. (c) EvoSuite, comparacion con Randoop

```
JAVA_HOME=$(/usr/libexec/java_home -v 11) PATH=$JAVA_HOME/bin:
    $PATH \
    ./gen-evo.sh assignment8_exercises.fail2ban.Server 30
```

Resultados (reportados por EvoSuite):

- Tests generados: 30, longitud total 111.
- Branch coverage: **89%** (48/54).
- Weak mutation coverage: **92%** (65/71).

- Mutation score: **72 %**.

EvoSuite tambien produjo `Server_Failed_ESTest`, con escenarios que rompen el estado interno (e.g. dejando `bans = null` desde tests del mismo paquete) y disparan excepciones no declaradas.

Comparacion.

	Randoop	EvoSuite
Branch coverage	3,85 %	89 %
Mutation score	4 % (PIT)	72 % (metrica propia)

Lectura. EvoSuite ejercita mucho mejor la logica de **Server**: su generacion guiada por cobertura construye secuencias que llegan a ramas profundas (multiples bans, expiraciones, interaccion con la lista ordenada), mientras que Randoop se queda en configuraciones superficiales.

5.5. (d) Usar `repOK()` para depurar con EvoSuite

Si, pero con un matiz:

- **Randoop** lee la anotacion `@CheckRep` y evalua el invariante durante la generacion. Cualquier violacion se reporta como `ErrorTest`.
- **EvoSuite** no toma `@CheckRep` automaticamente. Para usar `repOK()` como oraculo hay que:
 - agregar aserciones `assert obj.repOK();` en los tests generados (manual o via post-procesado), o
 - envolver las operaciones publicas para que invoquen `repOK()` antes y despues.

Conclusion. `repOK()` sigue siendo util para depurar con EvoSuite; solo que su integracion requiere un paso manual adicional.

5.6. (e) Propiedad jqwik sobre update con un unico generador

Se agrego `ServerPropertyTest.java`:

```
@Property(tries = 100)
void updateRemovesExactlyExpiredBans(@ForAll("updateScenarios")
    UpdateScenario scenario) {
    Server server = new Server();
    MutableTime time = new MutableTime(scenario.startTime);
    server.setTime(time);
```

```

    long[] expires = new long[scenario.ips.size()];
    for (int i = 0; i < scenario.ips.size(); i++) {
        IP ip = scenario.ips.get(i);
        assertTrue(server.addBan(ip));
        expires[i] = time.getCurrentTime() + BAN_WINDOW_MS;
        time.setNow(time.getCurrentTime() + 1L);
    }

    long nowAtUpdate = scenario.startTime + scenario.ips.size() +
        scenario.elapsed;
    time.setNow(nowAtUpdate);
    server.update();

    for (int i = 0; i < scenario.ips.size(); i++) {
        boolean banStillActive = expires[i] > nowAtUpdate;
        assertEquals(!banStillActive, server.connect(scenario.ips
            .get(i)));
    }
    assertTrue(server.repOK());
}

@Provide
Arbitrary<UpdateScenario> updateScenarios() {
    Arbitrary<IP> ip = Combinators.combine(
        Arbitraries.integers().between(0, 255),
        Arbitraries.integers().between(0, 255),
        Arbitraries.integers().between(0, 255),
        Arbitraries.integers().between(0, 255)).as(IP::new);

    Arbitrary<List<IP>> ips = ip.list().uniqueElements().
        ofMinSize(1).ofMaxSize(6);
    Arbitrary<Long> startTime = Arbitraries.longs().between(1
        _000_000L, 10_000_000L);
    Arbitrary<Long> elapsed = Arbitraries.longs().between(0L,
        120_000L);

    return Combinators.combine(ips, startTime, elapsed).as(
        UpdateScenario::new);
}

```

Que verifica.

- Después de `update()`, cada IP queda permitida o bloqueada según si su expiración ya pasó.
- El estado final del servidor sigue cumpliendo `repOK()`.

Por que un solo generador. El enunciado pide un *unico* generador. Empaquetamos todo el escenario (lista de IPs, tiempo inicial, elapsed) en un solo objeto `UpdateScenario` construido con `Combinators.combine(...)`, de modo que `jqwik` explore combinaciones consistentes en una sola dimensión del espacio de prueba.

Ejecucion:

```
JAVA_HOME=$(/usr/libexec/java_home -v 17) \
  mvn -q -Djacoco.skip=true \
  -Dtest=assignment8_exercises.fail2ban.ServerPropertyTest test
```

Resultado: la propiedad se cumple en las 100 iteraciones; el test queda en verde.

5.7. Resumen del ejercicio 5

- `repOK()` implementado y conectado al flujo de testing automático.
- Randoop ayudo a detectar y depurar defectos funcionales y de robustez tanto en `Server` como en las listas auxiliares, aunque su mutation score y cobertura quedaron bajos.
- EvoSuite logro mejor exploración de ramas y mutation score significativamente superior.
- La parte de PBT cierra con una propiedad `jqwik` para `update` apoyada en un unico generador.

Conclusiones

- **Randoop** es excelente como *primera linea*: rapido, requiere casi nada de setup y suele exponer defectos obvios (campos sin inicializar, NPEs en operaciones publicas, contratos mal implementados). En cambio, sobre clases con dependencias externas fuertes o estado profundo, su exploración aleatoria se queda corta.
- **EvoSuite** construye suites mas profundas porque su búsqueda evolutiva optimiza directamente la cobertura. Paga el costo en complejidad de integración (requiere un JDK compatible y su runtime puede ser fragil).
- **Mocking** con **EasyMock** es la herramienta natural cuando se quiere testear una clase *aislada* de sus colaboradores: programar el doble es mucho mas practico que armar el entorno real.

- **Property-Based Testing con jqwik** permite expresar el comportamiento esperado en abstracto (“despues de `update()`, cada IP esta donde debe estar”), y el framework lo verifica sobre cientos de escenarios distintos.
- Las tres tecnicas son *complementarias*, no excluyentes. Una buena suite las combina: PBT para invariantes, generacion automatica para descubrir casos, mocks para unidades aisladas.